Jacob Celniker

Assignment 2: Training and Visualizing a CNN on CIFAR

Model Architecture

After several experiments with extremities of both very shallow and very deep networks, a compromise was found in a feature extraction network of 6 convolutional layers. After each convolution, batch normalization was performed for increased training stability. After batch normalization, the rectified linear unit (ReLU) activation function was applied. The common practice of increasing the depth of filter map by factor of two was employed, starting with 12. A design target of 4x4 input height and width was selected for input to the classification layers. Max pooling layers were added after the activations of convolutional layers 4 and 6 to shrink the feature map to this end.

Table 1: Summary of Feature Extraction Architecture.

Layer	Input Volume	Filters	Kernel Width	Padding	Stride	Activation	Normalization
conv1	32x32x3	12	5	1	1	ReLU	Batch Norm
conv2	30x30x12	24	3	0	1	ReLU	Batch Norm
conv3	28x28x24	48	5	1	1	ReLU	Batch Norm
conv4	26x26x48	96	3	0	1	ReLU	Batch Norm
Max Pool	24x24x96	1	2	0	2	N/A	N/A
conv5	12x12x96	192	3	0	1	ReLU	Batch Norm
conv6	10x10x192	384	3	0	1	ReLU	Batch Norm
Max Pool	8x8x384	1	2	0	2	N/A	N/A

The first layer of the fully connected classification block was designed to conform to the output volume of the feature extraction layer and have 512 nodes in the second layer. The final output of the feature extraction block consisted of 4x4x384 volume, flattening out to 6144. A dropout of 0.3 was employed to reduce the propensity to overfit.

Table 2: Summary of Classification Architecture.

Layer	Nodes	Dropout
FC 1	6144	0.3
FC 2	512	0.3

Training Setup

Cross entropy loss was selected for training the CNN. For optimization, adam was used while experimenting with different feature extraction architectures. Once a feature extraction architecture was found which resulted in acceptable loss convergence, the optimizer was switched over to a parameterized SGD. This allowed for fine tuning the optimizer, ending up with implementation of a cosine learning rate scheduler. This setup allows for a learning rate which starts high and slowly decreases at first, then drops in a smooth sinusoidal trajectory towards the end of training. In the later epochs, the decrease of the learning rate epoch to epoch is again very gradual. Other parameters specified for SGD were momentum of 0.9 and weight decay of 0.0005. The initial learning rate of the cosine annealing learning rate scheduler is a key parameter. This is closely related to batch size and must be adjusted in coordination. After several full training iterations with 10, 128, 96, and 32, batch size of 32 and an initial learning rate of 0.025 were settled on.

Epochs of 250, 500, and 1000 were experimented upon with 1000 being selected for the final model. With the smaller numbers of epochs, loss convergence was still occurring, so the longer epochs proved to be beneficial. With the learning rate scheduler, the decrease of the learning rate is a function of the epoch count, so lower epochs meant a very sharp decrease in the middle parts of training and more epochs allowed for increased benefit along each stage of the learning rate curve.

Extensive data augmentation was employed to reduce the chance of overfitting and generalizing the model as much as possible with the given dataset. Random cropping with padding of 4 creates small spatial shifts in the training images. Random horizontal flipping with a chance of 0.5 was also used to augment the dataset. Affining, similar to cropping, was also applied. This process not only shifts the images randomly width and height wise by up to 10% but allows random zooming in and out of up to 10% as well. Another form of augmentation with a strong regularization effect is random erasing. This selects a randomly sized rectangular chunk of an image and replaces it with a random pigment. This process was parameterized with a 25% chance of occurring, a block size of between 2 and 20 percent of the image, and an aspect ratio of the erased portion between 0.3 and 3.3. This process blocks portions randomly to force the model to not rely too heavily upon

certain features, because in the test environment, you won't always see the full object in question.

All data augmentation processes increase regularization. In addition to those, dropout on the fully connected classification layers and global weight decay of 0.0005 encourage generalization. Batch normalization performed between convolution and activation helps reduce vanishing and exploding gradients, so dominant patterns don't push everything else out, aiding in regularization.

Training Results

The final model's training curve is shown below in figure 1. To illustrate the effect of the cosine annealing learning rate scheduler, the learning rate is shown as well. Past 800 epochs, validation loss began to flatten while training loss continued convergence. Final validation loss was 0.241 and final training loss was 0.096.

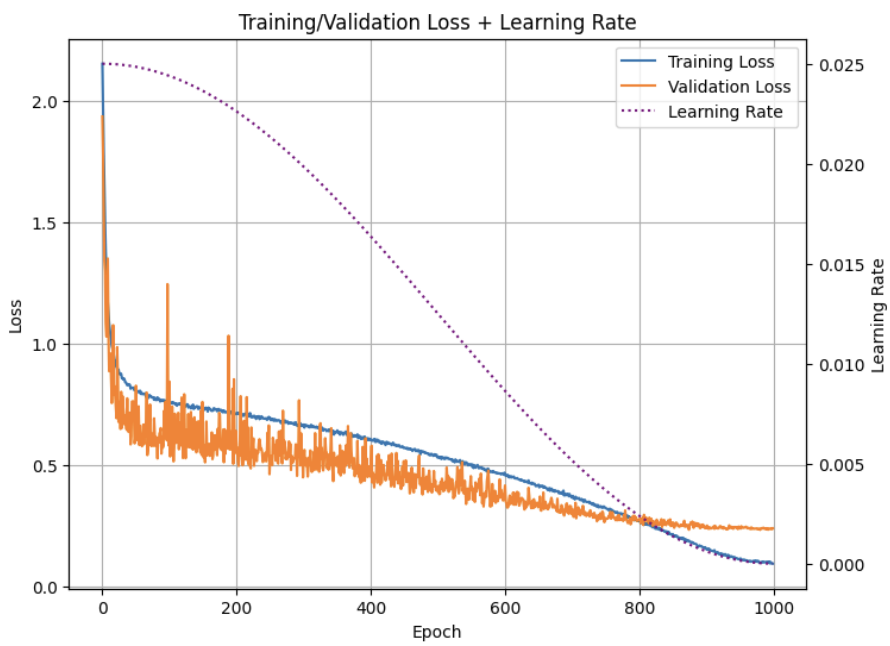

Fig. 1. Training and Validation Loss Overlayed with Learning Rate.

After experimenting with model architecture, optimizer settings, augmentation, and regularization parameters, performance well exceeds minimum requirements of ~65% with 92% overall test accuracy.

Table 3: Test Results of Final Model.

Class	Test Results	Percent Accuracy
Airplane	953/1000	95%
Automobile	967/1000	96%
Bird	888/1000	88%
Cat	856/1000	85%
Deer	937/1000	93%
Dog	890/1000	89%
Frog	955/1000	95%
Horse	952/1000	95%
Ship	955/1000	95%
Truck	945/1000	94%
Overall	9298/10000	92%

Early Layer Feature Map Visualization

A great way to understand convolution is to visualize the feature maps created by the filters. This way we can see which features are contributing most to correct classification downstream. It's also interesting to visualize the kernels themselves which contribute to the feature maps. In figures 2, 3, and 4, test images were passed through the first convolutional layer of the trained final model. The 12 convolutional feature maps are presented alongside the three kernels (corresponding to red, green, and blue input channels) for each of the output channels.

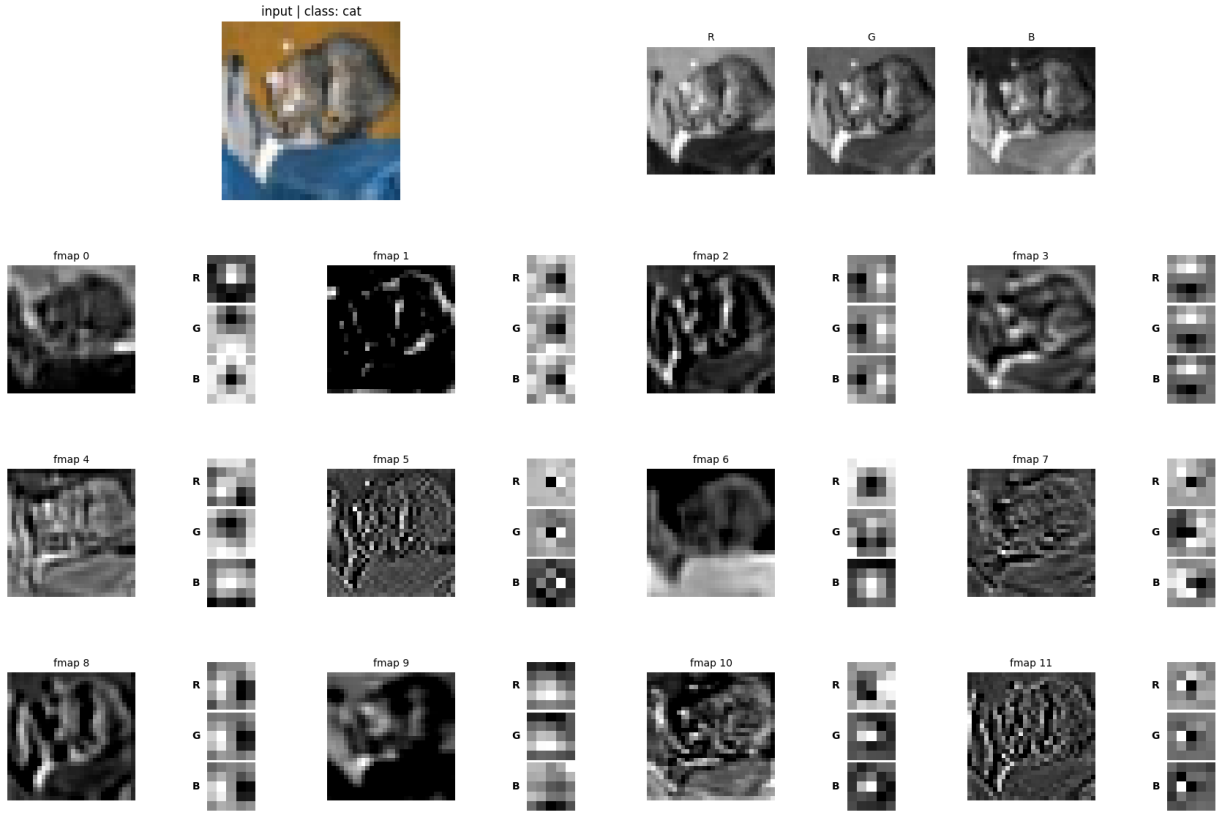

Fig. 2. Conv1 feature maps and associated kernels for an image from cat class.

By inspection of the kernels, it appears that filters 3 and 9 are bringing out horizontal edges, and filters 8 and 2 are filtering for vertical edges. Others like 1, 5, 10, and 11 seem to be aiming for sharp gradients bringing out edges in all directions. Others like 6, 7, and 9 look like they are bringing out textural aspects. Looking at the difference between the three color channels' kernels, interestingly the edge detecting filters (2, 3, 8) are uniform along all three colors, and the texture filters (6, 7, 9) have significant differences between them.

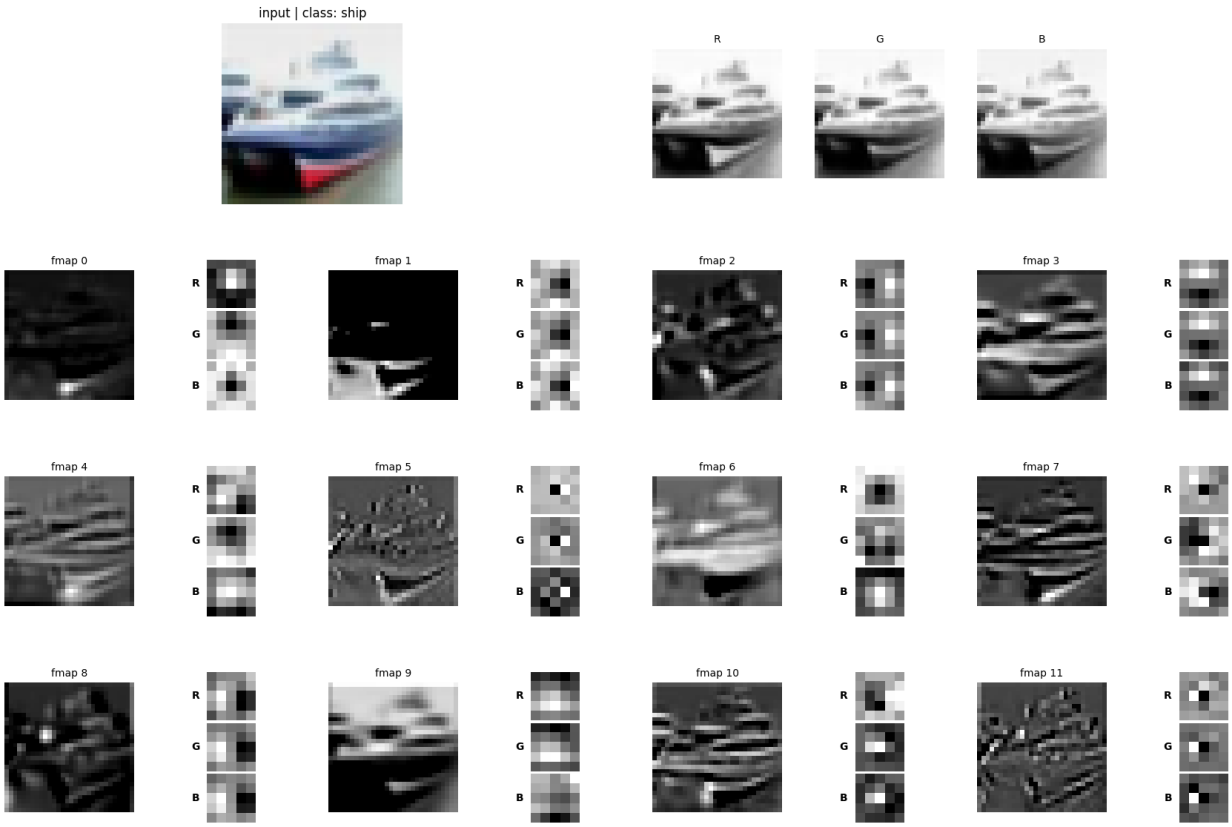

Fig. 3. Conv1 feature maps and associated kernels for an image from ship class.

In figure 3, we can see how the same 12 filters respond to an input from another class. Edge detecting filters have predictable responses. Texture filters are more ambiguous and unpredictable. Sharper texter filters like 5 and 11 give grainy snapshots of the input which would be almost unrecognizable alone. Smoother textures like 4 and 9 give blurry feature maps but retain general shapes. Also notable is that filters 0 and 6 almost look like inverses, particularly for the red and blue kernel. It follows that the feature map output by those respective filters look like opposites with fmap 0 very dark and inactivated and fmap 6 looking very bright all around.

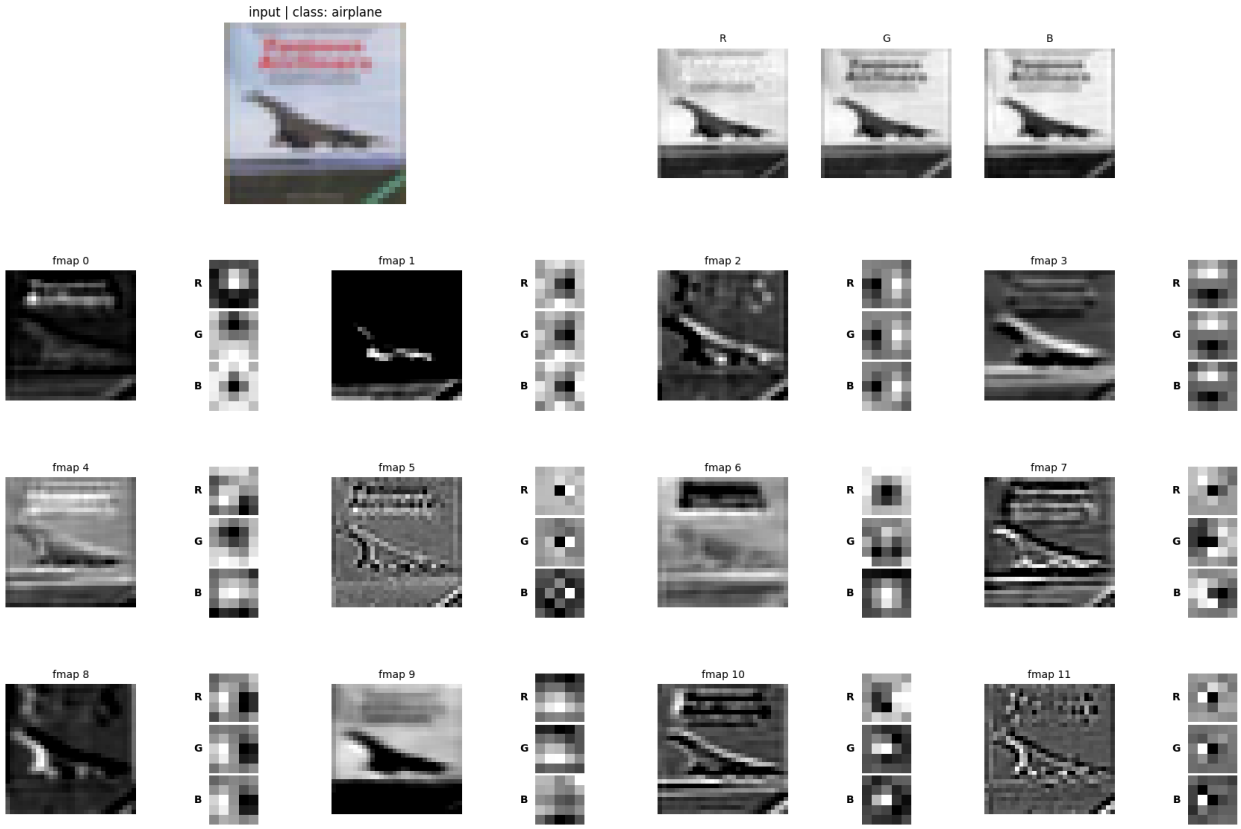

Fig. 4. Conv1 feature maps and associated kernels for an image from airplane class.

Figure 4 shows the same process but now with an airplane. This image has a very distinct object in the middle of the frame with a clear background. This makes the edge detecting filters really show their effectiveness and brings out which direction edge they are most suitable for. Again, we see an inverse pattern across fmaps 0 and 6 and grainy outputs for the sharp textures of 5 and 11. One thing very clear in this image but not figs 3 and 4 is that fmap 9 is very well suited for activating broad shapes like the whole body of the aircraft. This makes sense considering the nature of the kernels associated with that filter. They have very shallow gradients with high similarity between the red and green channel, while not being an edge detector.

Maximally Activating Images

For this analysis, one filter from the second convolutional layer was selected at a time and its internal kernels are visualized. Then the test images which result in the highest mean activation are displayed. This allows for analyzing which commonalties the maximally activated images might have, explaining why that filter was highly stimulating. From the last section, there were 12 filters each combining three internal kernels from the input space, so now there are 12 internal kernels associated with each filter.

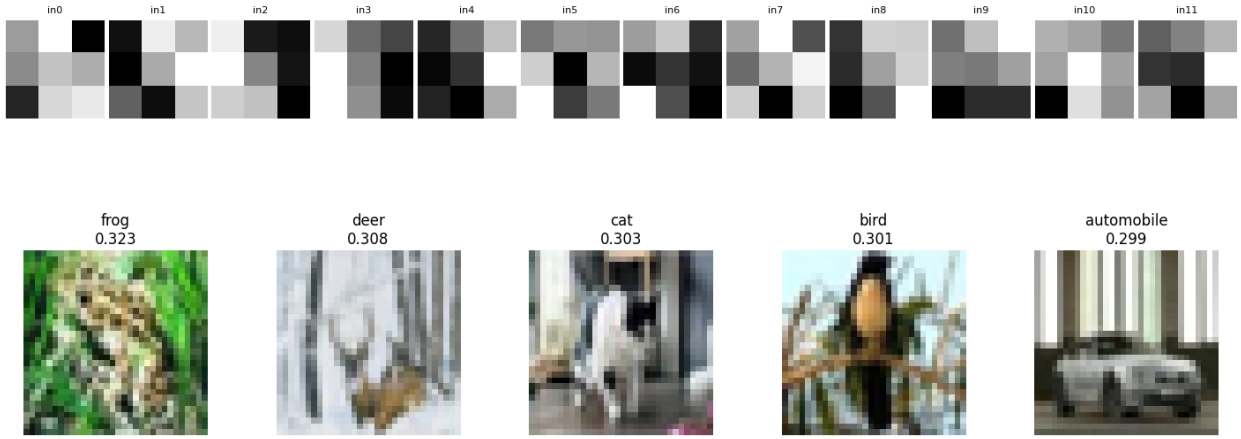

Fig. 5. Convolutional layer 2, filter 0, top 5 mean activated images.

The top 5 images activated by conv2, filter 0 are all from different classes. This might suggest that the primary selection here is not for abstract features but still for spatial patterns. It's difficult to see a correlation across the 5 images, but one thing is that they appear to favor thin lines running across most of the map. The lines on the frog's back and the grass behind it. The trees in the case of the deer, the lining of the doors behind the car, the branches in the case of the bird, and the very clear vertical lines in the case of the car. This is a pretty general feature that doesn't determine class very well; there could be long thin lines in images of just about anything, which might explain why the top activated images are all from different classes.

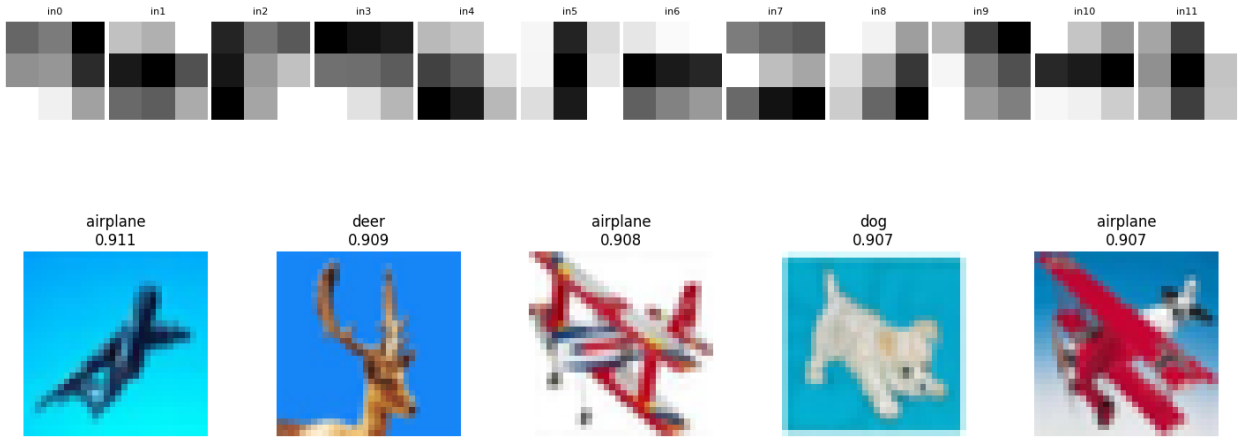

Fig. 6. Convolutional layer 2, filter 1, top 5 mean activated images.

Figure 6 shows the top activated images for conv2, filter 1. Right away, the most distinctive attribute is the level of activation. The top mean activations for this filter are all above 0.9. For filters 0 and 3, they are down around 0.2 and 0.3. These filters are highly specialized to bring out very clear, simple features, and it is present in the images selected. They all have incredibly strong, long lines on a uniform background. This can also be easily seen in the kernels themselves with very sharp vertical and horizontal lines appearing in nearly all of them. Unsurprisingly, the images with the strongest lines came mostly from a single class. The deer and dog image also appear in the top five because of their plain backgrounds and long distinct edges.

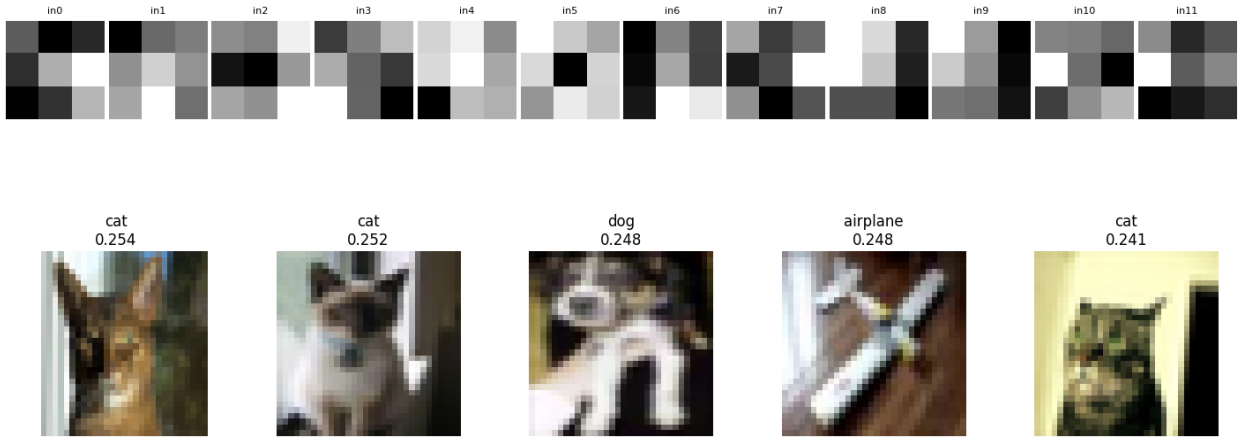

Fig. 7. Convolutional layer 2, filter 2, top 5 mean activated images.

In figure 7, we see a similar pattern from fig 6, with three out of five of the top activated images appearing from the same class. A dog and an airplane also make their way in due to similarity to the patterns seen in the cats, notably the shape and size of appendages.

Discussion and Reflection

I did as much as I could to get the highest test accuracy possible, so that the rest of the analysis was as meaningful as it could be. If the accuracy was at 50%, then the insight done on the kernels and feature maps would be less credible. Maybe with a poor performing classification model, there could be patterns in the feature space and the filters which are interesting, but don't represent the nature of a good performing model. I took much effort to create a good performing model so that I can study the properties of a model and trust that these properties are positive for performance and noteworthy.

In the pursuit of a good model, the first thing to nail down was the architecture, but what really improved the performance was data augmentation and optimization. Aggressive regularization like the augmentation algorithms and 0.3 percent dropout allowed lots of convergence before overfitting became a problem. Something which surprised me was the effect of the batch size. Higher batch size trains faster and converges faster at first, but plateaus earlier. Brining the batch size back down to 32 and a carefully tuned optimizer with a scheduled learning rate finally brought accuracy over 90%.